

Exercícios Complementares

Exercício 1. Família de Processos.

Crie um programa em C que utilize processos encadeados para imprimir no ecrã a sua própria 'árvore genealógica'. O output deverá respeitar a seguinte estrutura e ordem:

```
Pai ...  
└─ Filho ...  
    └─ Neto ...  
        └─ Bisneto ...
```

A criação dos processos deve ser encadeada e cada processo pode criar apenas um filho. Cada linha deve ser escrita por um processo diferente e deve mostrar a posição do processo na árvore (i.e., Pai, Filho, etc.), o PID e o PPID.

Exercício 2. Trabalho em equipa.

Dado um array com 100 mil números inteiros, pretende-se criar um programa para calcular a soma dos seus elementos. Pode preencher o array como desejar.

O programa deverá solicitar ao utilizador que indique o número de tarefas (threads) que vão colaborar na soma dos elementos do array (cada tarefa ficará responsável por uma parte). Deverá então dividir o array pelas N threads e cada uma irá calcular uma soma parcial. A thread principal calculará a soma total.

- Compare o tempo de execução para a utilização de 1 vs várias threads (rever: invocação do programa recorrendo a 'time').
- O facto de usar mais threads implica uma execução mais rápida? Justifique.

Exercício 3. Múltiplos Trincos.

Considere 2 tarefas (threads) que recorrem a 2 mutexs para aceder a uma secção crítica:

```
void* threadA(void* arg) {  
    printf("Thread A: tentar bloquear m1\n");  
    pthread_mutex_lock(&m1);  
    printf("Thread A: bloqueou m1\n");  
    sleep(1);  
    printf("Thread A: tentar bloquear m2\n");  
    pthread_mutex_lock(&m2);  
    printf("Thread A: entrou na secção crítica\n");  
    pthread_mutex_unlock(&m2);  
    pthread_mutex_unlock(&m1);  
    pthread_exit(NULL);  
    return NULL;  
}
```

```

void* threadB(void* arg) {
    printf("\t\t\t\t\tThread B: tentar bloquear m2\n");
    pthread_mutex_lock(&m2);
    printf("\t\t\t\t\tThread B: bloqueou m2\n");
    sleep(1);
    printf("\t\t\t\t\tThread B: tentar bloquear m1\n");
    pthread_mutex_lock(&m1);
    printf("\t\t\t\t\tThread B: entrou na secção crítica\n");
    pthread_mutex_unlock(&m1);
    pthread_mutex_unlock(&m2);
    pthread_exit(NULL);
    return NULL;
}

```

- Explique qual o problema do código apresentado e por que razão ocorre. Que 'regra' deve ser seguida nestes casos?
- Corrija o código e implemente uma solução segura. Deve manter a premissa de cada thread ter de bloquear obrigatoriamente os 2 mutexs antes de entrar na secção crítica.

Exercício 4. Chat Unidirecional com Memória Partilhada.

Pretende-se criar 2 programas independentes: 1 emissor e 1 recetor. Os programas devem comunicar entre si recorrendo a memória partilhada.

Quando o utilizador introduz uma mensagem no emissor e pressiona 'ENTER', o recetor deverá mostrar essa mensagem no output (passados 3 segundos).

Para melhor visualização do output, pode executar os 2 programas e colocar as janelas de execução lado a lado.

- Implemente e teste o programa descrito.
- Desenvolva outra versão do programa, mas agora com 3 recetores. Neste caso, apenas 1 dos recetores conseguirá ler e imprimir cada mensagem enviada.

Exercício 5. Algoritmos de Escalonamento.

Considere os seguintes processos e respetivos tempos de chegada e de execução (burst):

Processo	Chegada	Burst
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Para cada um dos algoritmos em baixo, indique qual seria a **ordem de execução** dos processos, o **tempo médio de espera**, o **turnaround** e conclua sobre qual o algoritmo mais adequado neste caso. Justifique.

- a) FCFS
- b) SJF (preemptivo e não preemptivo)
- c) Round Robin (q=3)

Exercício 6. Sistema de Gestão de Processos.

Desenvolva um programa que simula um sistema de execução de processos utilizando uma fila de espera. O programa deve implementar o algoritmo de escalonamento Round Robin (e.g., QUANTUM = 2), garantindo que cada processo recebe um tempo de CPU de forma cíclica.

Cada processo deverá ser representado por uma estrutura de dados que armazene o seu estado (e.g., pid, burst, burst_restante e estado).

O sistema deverá ainda incluir 2 threads para representar 2 cores do CPU – responsáveis por executar os processos. Considere também a utilização de mecanismos de sincronização no acesso à fila de processos.